Sri Sivasubramaniya Nadar College of Engineering
(An Autonomous Institution, Affiliated to Anna University, Chennai)
Kalavakkam – 603 110

Department of Computer Science & Engineering

Computer Networks Laboratory

1513 – Computer Networks Lab

Assignment 2

Multi-Client Chat Application using TCP and UDP Sockets

Name:	Simiyon Vinscent Samuel L
Register Number:	3122247001062
Year / Semester:	III Year / V Semester
Course:	1513 Computer Networks Lab
Date of Submission:	06 March 2026

1. Objective

To design and implement client–server applications in C using socket programming that allow students to retrieve academic information (stored in a CSV file `students.csv`) from a server, using both TCP (connection-oriented) and UDP (connectionless) transport, and to compare the two implementations based on connection setup, reliability, ordering, error recovery, and communication overhead.

- **TCP application:** retrieve the complete academic profile of a student given a Roll Number, with the server handling multiple clients concurrently using POSIX threads.
- **UDP application:** verify a student record (Roll Number and Name only) using a single connectionless datagram exchange.

2. Server Algorithm

2.1 TCP Server

1. Create a TCP socket using `socket(AF_INET, SOCK_STREAM, 0)`.
2. Set `SO_REUSEADDR` and bind the socket to port 8080 using `bind()`.
3. Call `listen()` to mark the socket as passive and wait for connections.
4. In an infinite loop, `accept()` an incoming client connection.
5. Spawn a new POSIX thread (`pthread_create`) per client so multiple clients are served concurrently; detach the thread.
6. Inside the thread: `recv()` the Roll Number, linearly search `students.csv` for a matching record.
7. If found, format and `send()` the full record (Roll No, Name, Department, Semester, CGPA); otherwise send `"Student Record Not Found"`.
8. Close the client socket and terminate the thread.

2.2 UDP Server

1. Create a UDP socket using `socket(AF_INET, SOCK_DGRAM, 0)`.
2. Bind the socket to port 8081 using `bind()`.
3. In an infinite loop, block on `recvfrom()` to receive a Roll Number datagram together with the client's address.
4. Search `students.csv` for the Roll Number.
5. If found, build a short reply containing only Roll Number and Name; otherwise reply `"Student Record Not Found"`.
6. Send the reply back to the client's address using `sendto()` (no connection state is kept).

3. Client Algorithm

3.1 TCP Client

1. Create a TCP socket and `connect()` to the server at `127.0.0.1:8080`.
2. Prompt the user to enter a Roll Number and `send()` it to the server.

3. Loop on `recv()` until the connection is closed, printing the received bytes.
4. Close the socket.

3.2 UDP Client

1. Create a UDP socket (no `connect()` required).
2. Prompt the user for a Roll Number and `sendto()` it to `127.0.0.1:8081`.
3. Block on `recvfrom()` to receive the server's reply.
4. Display the reply and close the socket.

4. Server Code

4.1 tcp_server.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <pthread.h>

#define PORT 8080
#define CSV_FILE "students.csv"

typedef struct {
    int socket;
} client_t;

int search_student(const char *roll, char *response) {
    FILE *file = fopen(CSV_FILE, "r");
    if (!file) {
        strcpy(response, "Error: Database file not found.\n");
        return 0;
    }

    char line[256];
    char temp_line[256];
    int found = 0;

    while (fgets(line, sizeof(line), file)) {
        line[strcspn(line, "\r\n")] = 0;
        strcpy(temp_line, line);

        char *token = strtok(temp_line, ",");
        if (token && strcmp(token, roll) == 0) {
            strcpy(temp_line, line);
            char *r = strtok(temp_line, ",");
            char *n = strtok(NULL, ",");
            char *d = strtok(NULL, ",");
            char *s = strtok(NULL, ",");
            char *c = strtok(NULL, ",");

            sprintf(response, "Roll Number : %s\nName        : %s\nDepartment
                : %s\nSemester    : %s\nCGPA        : %s\n",
                    r ? r : "N/A",
                    n ? n : "N/A",
                    d ? d : "N/A",
```

```
                    s ? s : "N/A",
                    c ? c : "N/A");
            found = 1;
            break;
        }
    }

    fclose(file);
    if (!found) {
        strcpy(response, "Student Record Not Found\n");
    }
    return found;
}

void *client_handler(void *arg) {
    client_t *client = (client_t *)arg;
    int sock = client->socket;
    free(client);

    char buffer[1024];
    memset(buffer, 0, sizeof(buffer));

    int bytes_read = recv(sock, buffer, sizeof(buffer) - 1, 0);
    if (bytes_read > 0) {
        // Strip trailing whitespaces or newlines
        buffer[strcspn(buffer, "\r\n ")] = 0;
        printf("[TCP Server] Received request for Roll Number: '%s'\n", buffer)
            ;

        char response[1024];
        search_student(buffer, response);

        send(sock, response, strlen(response), 0);
    }

    close(sock);
    return NULL;
}

int main() {
    int server_fd, client_sock;
    struct sockaddr_in address;
    int addrlen = sizeof(address);

    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }

    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("Bind failed");
        close(server_fd);
        exit(EXIT_FAILURE);
    }

    if (listen(server_fd, 10) < 0) {
```

```
        perror("Listen failed");
        close(server_fd);
        exit(EXIT_FAILURE);
    }

    printf("[TCP Server] Listening on port %d...\n", PORT);

    while (1) {
        client_sock = accept(server_fd, (struct sockaddr *)&address, (socklen_t
            *)&addrlen);
        if (client_sock < 0) {
            perror("Accept failed");
            continue;
        }

        pthread_t thread_id;
        client_t *client = malloc(sizeof(client_t));
        client->socket = client_sock;

        if (pthread_create(&thread_id, NULL, client_handler, (void*)client) <
            0) {
            perror("Thread creation failed");
            free(client);
            close(client_sock);
        }
        pthread_detach(thread_id);
    }

    close(server_fd);
    return 0;
}
```

4.2 udp_server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8081
#define CSV_FILE "students.csv"

int search_student_udp(const char *roll, char *response) {
    FILE *file = fopen(CSV_FILE, "r");
    if (!file) {
        strcpy(response, "Error: Database file not found.\n");
        return 0;
    }

    char line[256];
    char temp_line[256];
    int found = 0;

    while (fgets(line, sizeof(line), file)) {
        line[strcspn(line, "\r\n")] = 0;
        strcpy(temp_line, line);

        char *token = strtok(temp_line, ",");
        if (token && strcmp(token, roll) == 0) {
            char *name = strtok(NULL, ",");
```

```
            sprintf(response, "Roll Number : %s\nStudent Name: %s\n", token,
                name ? name : "N/A");
            found = 1;
            break;
        }
    }

    fclose(file);
    if (!found) {
        strcpy(response, "Student Record Not Found\n");
    }
    return found;
}

int main() {
    int sockfd;
    char buffer[1024];
    struct sockaddr_in servaddr, cliaddr;

    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }

    memset(&servaddr, 0, sizeof(servaddr));
    memset(&cliaddr, 0, sizeof(cliaddr));

    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = INADDR_ANY;
    servaddr.sin_port = htons(PORT);

    if (bind(sockfd, (const struct sockaddr *)&servaddr, sizeof(servaddr)) < 0)
        {
        perror("Bind failed");
        close(sockfd);
        exit(EXIT_FAILURE);
    }

    printf("[UDP Server] Listening on port %d...\n", PORT);

    socklen_t len = sizeof(cliaddr);
    int n;

    while (1) {
        memset(buffer, 0, sizeof(buffer));
        n = recvfrom(sockfd, buffer, sizeof(buffer) - 1, 0, (struct sockaddr *)
            &cliaddr, &len);
        if (n < 0) {
            perror("recvfrom failed");
            continue;
        }

        buffer[strcspn(buffer, "\r\n ")] = 0;
        printf("[UDP Server] Received request for Roll Number: '%s'\n", buffer)
            ;

        char response[1024];
        search_student_udp(buffer, response);

        sendto(sockfd, response, strlen(response), 0, (const struct sockaddr *)
            &cliaddr, len);
    }

```

```
    close(sockfd);
    return 0;
}
```

5. Client Code

5.1 tcp_client.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8080
#define SERVER_IP "127.0.0.1"

int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;

    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("Socket creation error");
        return -1;
    }

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    if (inet_pton(AF_INET, SERVER_IP, &serv_addr.sin_addr) <= 0) {
        perror("Invalid address/ Address not supported");
        close(sock);
        return -1;
    }

    if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
        perror("Connection Failed");
        close(sock);
        return -1;
    }

    char roll[100];
    printf("Enter Roll Number: ");
    if (fgets(roll, sizeof(roll), stdin) != NULL) {
        roll[strcspn(roll, "\n")] = 0;

        send(sock, roll, strlen(roll), 0);

        char buffer[1024] = {0};
        int bytes_read;
        printf("\n--- Student Details ---\n");
        while ((bytes_read = recv(sock, buffer, sizeof(buffer) - 1, 0)) > 0) {
            buffer[bytes_read] = '\0';
            printf("%s", buffer);
        }
        printf("-----------------------\n");
    }

    close(sock);
    return 0;
}
```

5.2 udp_client.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8081
#define SERVER_IP "127.0.0.1"

int main() {
    int sockfd;
    char buffer[1024];
    struct sockaddr_in servaddr;

    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }

    memset(&servaddr, 0, sizeof(servaddr));

    servaddr.sin_family = AF_INET;
    servaddr.sin_port = htons(PORT);
    if (inet_pton(AF_INET, SERVER_IP, &servaddr.sin_addr) <= 0) {
        perror("Invalid address/ Address not supported");
        close(sockfd);
        exit(EXIT_FAILURE);
    }

    char roll[100];
    printf("Enter Roll Number: ");
    if (fgets(roll, sizeof(roll), stdin) != NULL) {
        roll[strcspn(roll, "\n")] = 0;

        socklen_t len = sizeof(servaddr);

        sendto(sockfd, roll, strlen(roll), 0, (const struct sockaddr *)&
            servaddr, len);

        memset(buffer, 0, sizeof(buffer));
        int n = recvfrom(sockfd, buffer, sizeof(buffer) - 1, 0, (struct
            sockaddr *)&servaddr, &len);
        if (n >= 0) {
            buffer[n] = '\0';
            printf("\n--- Student Details ---\n");
            printf("%s", buffer);
            printf("-------------------------\n");
        } else {
            perror("Receive failed");
        }
    }

    close(sockfd);
    return 0;
}
```

6. Output / Test Cases

Both servers were started in the background and the clients were run against Roll Numbers that exist in `students.csv` (e.g. 101, 102) as well as one that does not (999), to test both the success and *record-not-found* paths.

6.1 Screenshots

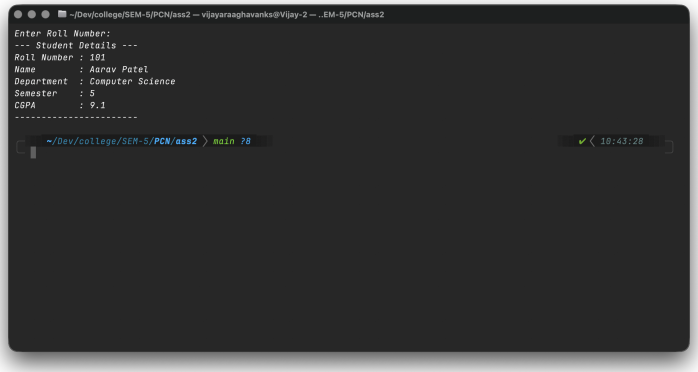


Figure 1: TCP client – record found (Roll Number 101)

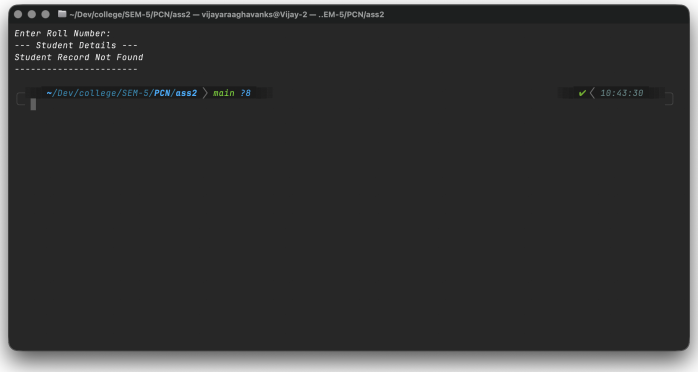


Figure 2: UDP client – record not found (Roll Number 999)

6.2 Console Output (all test cases)

```
=== TCP client: found (Roll 101) ===
Enter Roll Number:
--- Student Details ---
Roll Number : 101
Name        : Aarav Patel
Department  : Computer Science
Semester    : 5
CGPA        : 9.1
-----------------------

=== TCP client: not found (Roll 999) ===
Enter Roll Number:
--- Student Details ---
Student Record Not Found
-----------------------
```

```
=== UDP client: found (Roll 102) ===
Enter Roll Number:
--- Student Details ---
Roll Number : 102
Student Name: Diya Sharma
----------------------

=== UDP client: not found (Roll 999) ===
Enter Roll Number:
--- Student Details ---
Student Record Not Found
----------------------

=== TCP Server log ===
[TCP Server] Listening on port 8080...
[TCP Server] Received request for Roll Number: '101'
[TCP Server] Received request for Roll Number: '999'
[TCP Server] Received request for Roll Number: '101'
[TCP Server] Received request for Roll Number: '999'

=== UDP Server log ===
[UDP Server] Listening on port 8081...
[UDP Server] Received request for Roll Number: '102'
[UDP Server] Received request for Roll Number: '999'
[UDP Server] Received request for Roll Number: '102'
[UDP Server] Received request for Roll Number: '999'
```

7. Comparison: TCP vs UDP

Parameter	**TCP (1a)**	**UDP (1b)**
Connection	Connection-oriented; three-way handshake (`connect()`/`accept()`) sets up a dedicated channel.	Connectionless; each datagram is sent independently via `sendto()`.
Reliability	Reliable – acknowledgments, timeouts and retransmission guarantee delivery.	Unreliable – lost datagrams are silently dropped, no retransmission.
Ordering	Guaranteed in-order delivery via sequence numbers.	No ordering guarantee; datagrams may arrive out of order or be duplicated.
Error Recovery	Built-in checksum, duplicate/missing segment detection, flow & congestion control.	Only a simple checksum; corrupted packets are dropped with no recovery.
Overhead	High – 20-byte header, handshake and teardown packets, per-connection state.	Low – 8-byte header, stateless, minimal overhead.
Concurrency	Concurrent server using one `pthread` per client.	Iterative single-threaded loop; stateless processing needs no threads.

8. Learning Outcomes

- Understood how a TCP socket lifecycle (`socket`–`bind`–`listen`–`accept`) differs from the simpler UDP lifecycle (`socket`–`bind`–`recvfrom`/`sendto`).
- Learned to build a concurrent TCP server using POSIX threads so multiple clients can be served without blocking one another.
- Observed experimentally that TCP guarantees ordered, reliable delivery at the cost of connection-setup overhead, while UDP is lightweight but offers no delivery guarantee.
- Gained practical experience parsing a CSV-based record store and returning either full or partial records depending on protocol requirements.
- Understood why real-world protocols choose UDP for simple, latency-sensitive query/response exchanges and TCP for transfers that must be reliable and ordered.